

2026-06-10-execution-models

Execution-Model Families — Deep Web Research (SP6 feeder)

Date: 2026-06-10 **Author:** DEEP-WEB-RESEARCH subagent (§11.4.99 / §11.4.8 / §11.4.74)
Scope: Research feeder for SP6 (“other execution models / dynamic flows”). Read-only over the repo; sources verified against current official/authoritative docs + OSS repos (see footer). Where a claim could not be verified online it is marked HONEST-GAP. **Mandate quoted (operator):**
“We MUST support all ways of execution and development: dynamic flows and all other that exist. Perform deep web research and obtain details and technical knowledge and source code access for all of them then incorporate each fully into the System.”

Anti-bluff note (§11.4.6): every external claim below carries a source URL in the footer. License/language/status facts were fetched from the canonical repo pages, not memory. Items I could not confirm are flagged HONEST-GAP rather than asserted.

0. What HelixCode/HelixAgent already has (reuse baseline, §11.4.74)

Surveyed read-only on 2026-06-10. These are the in-house abstractions every recommendation below is measured against:

Own-org submodule / pkg	Module id	Current capability (from REA
submodules/agenti c	digital.vasic.agenti c	Graph-based agentic workflow State *WorkflowState, Node{ID,Type,Handler,C node types Agent / Tool / Subgraph; statuses Pending Failed; retries+backoff, period max-iteration guard, shared WorkflowState{Messages Graph is built up-front (bui AddEdge/mutate API was four
submodules/planning	digital.vasic.planning	HiPlan (hierarchical goal→mile milestones, adaptive), MCTS (U backprop), ToT (BFS/DFS/beam Pluggable Generator/Evaluator/L
submodules/ concurrency	digital.vasic.concurrency	Worker pool (pkg/ pool, incl. queue, Critical/High/.../Low), semaphore, resource monitor. Th every new runtime should sit on
submodules/ debate_orchestrator	digital.vasic.debate	Multi-LLM consensus/dissent or Orchestrator .ConductDe

Own-org submodule / pkg	Module id	Current capability (from REA
		ProviderInvoker, Lesson pkgs are honest stubs.
submodules/ llm_orchestrator	digital.vasic.llmorchestrator	Headless CLI-agent pool (Open protocol, capability matching, pe is the multi-agent transport/po
helix_code/ internal/workflow	(inner app)	Linear Workflow{Steps [] accessors under sync.RWMute subpkgs, chaos/stress tests. NOT
helix_code/ internal/task	(inner app)	DependencyManager (Vali CheckDependenciesComp + DB persistence, checkpoint, pr dependency / DAG-data layer c scheduler.

Net: HelixCode already has (a) a *static* agentic graph engine, (b) planning/search algorithms, (c) concurrency primitives, (d) task-dependency data + a linear executor, (e) multi-agent pools. The genuine gaps SP6 must fill are: **runtime-mutable graphs, durable/compensating orchestration, reactive/dataflow streaming, behavior-tree control, and a true DAG scheduler** over the existing concurrency substrate.

1. Dynamic flows / runtime-mutable workflow graphs

Definition. Execution models where the graph topology is decided/extended *at runtime* from state — conditional routing, dynamic fan-out (map-reduce), node-generated successors, and (in durable engines) replay-based resumption. Distinct from a graph fixed before Run().

Canonical OSS implementations. - **LangGraph** (langchain-ai/langgraph, Python + JS; MIT). Core: StateGraph(State), add_node, add_edge, add_conditional_edges; Command (a node returns *state update + next-node* in one value, collapsing routing into the node); Send (inside a conditional edge, dynamically invoke a node N times with custom per-branch state → native map-reduce). Runtime context injected via Runtime[Context]. This is the closest match to “dynamic flows.” - **Temporal** (temporalio/sdk-go, Go, MIT, v1.44.1 / 2026-05-28). Durable execution: deterministic Workflow functions + Activities; topology emerges from ordinary Go control flow and is made durable by **event-sourced replay** (no static graph at all). First-class Saga support (see §3). - **Prefect / Dagster / Flyte / Airflow** (Python; Apache-2.0 except Prefect Apache-2.0). Prefect builds the dependency graph *as code executes* (dynamic mapping, subflows). Dagster is asset-graph/lineage-centric. Airflow 3.2 (2026-04) added dynamic task mapping + asset partitioning; still primarily static DAG-as-code. Flyte = K8s-native typed DAG.

Core abstractions to lift. State (reducer-merged), node-returns-routing (Command), dynamic fan-out (Send), durable replay (Temporal), runtime dependency accretion (Prefect).

Map onto Go + HelixCode. Extend agentic with: (1) a runtime Graph.AddNode/ AddEdge/RemoveNode + a NodeResult{StatePatch, GoTo []NodeID} so a NodeHandler can both patch state and name successors (LangGraph Command); (2) a Fan(targetNode, []State) primitive over concurrency.Map for dynamic map-reduce (LangGraph Send); (3) a reducer-typed WorkflowState merge. Durable replay is a

separate concern → see §3 (`flow_engine` should wrap Temporal-style replay or integrate Temporal Go SDK directly rather than reimplement event-sourcing).

Distinct vs what we have / recommendation. Distinct = runtime topology mutation + Command/Send semantics + durable replay; we only have a static graph + linear executor. → **EXTEND** `agent` for runtime-mutable graph + Command/Send; **CREATE-NEW** `flow_engine` as the high-level dynamic-flow façade that composes `agent` + planning + (optionally) wraps Temporal Go SDK for durability. Do **not** reimplement Temporal's replay (§11.4.74) — wrap/depend on `temporalio/sdk-go` (Go, MIT) where durability is required.

2. Dataflow / reactive (ReactiveX, FBP, Apache Beam)

Definition. Computation as data moving through a network of operators. Two sub-families: **reactive streams** (push, observable/operator pipelines) and **flow-based programming/FBP** (named components with typed in/out ports connected by bounded channels). Beam adds a windowed batch+stream dataflow model.

Canonical OSS implementations. - **RxGo** (`ReactiveX/RxGo`, Go, MIT). Observables + operators implemented as channel-connected pipeline stages of goroutines (`Map/Filter/FlatMap/Reduce/backpressure` strategies). - **GoFlow** (`trustmaster/goflow`, Go, MIT). FBP: Component with input/output **ports** + state; the network is wired declaratively and runs asynchronously; data are `Packets`/IPs over channels. - **Apache Beam Go SDK** (`apache/beam/sdks/v2/go/pkg/beam`, Go, Apache-2.0). `PCollection` (bounded/unbounded), `ParDo` (per-element parallel fn), windowing; pluggable runners (Direct local, Dataflow, Flink...).

Core abstractions to lift. Observable + operator chain (Rx), typed-port Component + channel wiring (FBP), `PCollection/ParDo/windowing` (Beam).

Map onto Go + HelixCode. Go channels + `concurrency.pool/Map` are a natural substrate. A `pipeline_runtime` package should expose: `Stream[T]` (observable), composable operators (`Map/Filter/FlatMap/Window/Buffer/Reduce`), and a **Component/Port** FBP layer for declaratively wired agent pipelines. Backpressure → reuse `concurrency` rate-limiter/semaphore. Beam's runner abstraction is over-scoped for our needs (HONEST-GAP: full Beam parity = large; recommend Rx+FBP subset first).

Distinct vs what we have / recommendation. Distinct = push-based streaming operators + typed-port FBP wiring + windowing; we have none of this (our `streaming` submodule is LLM token streaming, not a dataflow operator graph — verify scope before reuse). → **CREATE-NEW** `pipeline_runtime` (Rx-style streams + FBP components) built on `concurrency`; optionally vendor RxGo/GoFlow patterns rather than full Beam.

3. Saga / compensation orchestration

Definition. Long-running distributed transaction as a sequence of local steps each with a registered **compensating action**; on failure, run compensations in reverse for completed steps (no global lock/2PC).

Canonical OSS implementations. - **Temporal Go SDK** (`temporalio/sdk-go`, Go, MIT). First-class saga: register compensations and run them on abort; the common Go idiom uses `defer` + a compensation stack. Samples: `temporalio/samples-go/saga/`

workflow.go. - **Community:** kevinmichaelchen/temporal-saga-grpc (Go) demonstrates cross-microservice saga rollback.

Core abstractions to lift. `Saga.AddCompensation(fn)`, `Saga.Compensate()` (reverse order), step-level idempotency, durable progress so compensations survive a crash.

Map onto Go + HelixCode. A saga package (could live in `flow_engine` or `dag_orchestrator`): `Saga{steps []Step{Do, Undo}}`, run forward collecting Undo closures, on error run Undo in LIFO. For *durable* sagas (survive process death) → depend on Temporal Go SDK rather than build our own event store. `agentic`'s checkpointing gives at-most weak durability — adequate for in-process compensation, not crash-durable.

Distinct vs what we have / recommendation. Distinct = explicit compensation/rollback semantics (we have retry+checkpoint but no compensation model). → **CREATE-NEW lightweight saga primitive** inside `flow_engine`; **REUSE** `temporalio/sdk-go` (Go, MIT) for crash-durable sagas instead of reimplementing durable execution (§11.4.74).

4. Behavior trees (agent control)

Definition. A directed rooted tree ticked each frame; **composite** nodes (Sequence = AND, Selector/Fallback = OR), **decorator** nodes (Inverter, Retry, Timeout, Repeat), **leaf** nodes (Action/Condition). Each tick returns Success/Failure/Running. Strong for reactive, interruptible agent control with explicit fallbacks.

Canonical OSS implementations. - joeycumines/go-behaviortree (**Go**; license HONEST-GAP — pkg.go.dev shows “redistributable” but the exact SPDX wasn't confirmed online; verify the repo LICENSE before vendoring). Stateless Tick funcs, Selector, Sequence, context-cancellation support. - askft/go-behave (**Go**, MIT). Build tree from nodes, `NewBehaviorTree(Config)`; extensible composite/decorator/leaf. - Reference (non-Go): `BehaviorTree.CPP` (the de-facto standard; C++).

Core abstractions to lift. `Node.Tick(ctx) Status{Success, Failure, Running}`, composites (Sequence/Selector/Parallel), decorators (Inverter/Retry/Timeout/Repeat), blackboard (shared state).

Map onto Go + HelixCode. A `behavior_tree` package: `Tick func(ctx) (Status, error)`, `Sequence/Selector/Parallel(children...)`, decorators, a Blackboard aliasing `agentic.WorkflowState.Variables`. BT leaves can wrap `agentic Tool/Agent` nodes → BT becomes a *control layer* over existing handlers. Distinct value: reactive re-evaluation + explicit fallback that the current condition-edge model expresses awkwardly.

Distinct vs what we have / recommendation. Distinct = tick-based reactive control with Running state + Selector fallbacks; `agentic` is run-to-completion graph traversal, no per-tick re-eval. → **CREATE-NEW** `behavior_tree` submodule (small, vendor patterns from `askft/go-behave` MIT); leaves delegate to `agentic/llm_orchestrator` handlers.

5. Pipeline runtimes (Unix-pipe / streaming / map-reduce)

Definition. Linear/branching stages connected by buffered channels; each stage transforms a stream; includes map-reduce (partition→map→shuffle→reduce) and Unix-pipe composition (stdout→stdin).

Canonical OSS implementations / patterns. Go's own **pipeline pattern** (Go blog "Pipelines and cancellation": stage = goroutine group over channels — this is exactly how RxGo §2 is built). Map-reduce = `concurrency.Map` + a reduce stage. Beam (§2) is the heavyweight version.

Core abstractions to lift. Stage `func(in <-chan T) <-chan U`, fan-out/fan-in, cancellation via `context`, bounded buffers for backpressure.

Map onto Go + HelixCode. This is the **lowest-friction** family — Go channels + `concurrency.pool.Map` already cover ~80%. A thin pipeline API (`Pipe(stages...)`, `FanOut(n)`, `FanIn()`, `MapReduce(mapper, reducer)`) belongs in the same `pipeline_runtime` package as §2 (Rx/FBP and Unix-pipe are the same substrate at different abstraction levels).

Distinct vs what we have / recommendation. Mostly already expressible via `concurrency`; distinct = the ergonomic stage/fan-out/fan-in API + map-reduce helper. → **EXTEND** `concurrency` with a `pipeline_subpkg` OR fold into `pipeline_runtime` (§2). Reuse-heavy, minimal new code.

6. DAG orchestration patterns + scheduling

Definition. Schedule a directed-acyclic graph of tasks honoring dependencies: topological order, ready-set dispatch, max-parallelism, retries/backoff, failure-propagation policies (fail-fast vs continue), and (dynamic DAGs) runtime task mapping.

Canonical OSS implementations. Airflow (Apache-2.0; static DAG-as-code + dynamic task mapping since 2.3, asset partitioning 3.2/2026-04), Prefect (Apache-2.0; runtime-built graph), Dagster (Apache-2.0; asset graph + lineage), Flyte (Apache-2.0; typed K8s DAG), Temporal (durable, §3). Go-native general DAG schedulers are comparatively rare (HONEST-GAP: no single dominant Go DAG-scheduler library surfaced — most Go usage embeds Temporal or hand-rolls topo-sort over a worker pool).

Core abstractions to lift. `DAG{nodes, edges}` + topo-sort, ready-queue scheduler, per-node retry/timeout, failure policy, dynamic-mapping (expand one node into N at runtime — bridges to §1 Send), result/lineage tracking (Dagster).

Map onto Go + HelixCode. We already have the *data* (`task.DependencyManager` validates deps + completion). The missing piece is a **scheduler**: a `dag_orchestrator` that consumes a DAG (built from task dependencies or an *agentic* graph), computes the ready-set, dispatches via `concurrency.pool` honoring a parallelism cap + `concurrency.queue` priority, applies retry/backoff + failure policy, and records lineage. Dynamic DAGs = let a completed node emit new nodes (composes with §1).

Distinct vs what we have / recommendation. Distinct = the *scheduler* (ready-set dispatch + parallelism + failure policy + lineage); we have dependency-validation but no scheduler. → **CREATE-NEW** `dag_orchestrator` over `concurrency` (pool+queue) + task (dependency data); **REUSE Temporal** only where crash-durability/distribution is required.

7. Agent-mesh / swarm coordination

Definition. Multiple autonomous agents collaborate via handoffs, shared message bus / group chat, role specialization, or topology (star/mesh/hierarchy). Routing decisions are themselves agentic.

Canonical OSS implementations. - **OpenAI Swarm** (openai/swarm, Python, MIT) — **DEPRECATED**, superseded by **OpenAI Agents SDK** (openai/openai-agents-python, Python, MIT, prod, 2025-03). Primitives: **Agent** (instructions+tools) + **handoff** (an agent returns another Agent to transfer control). Lightweight, stateless-between-calls. - **CrewAI** (Python, MIT) — role/goal/backstory agents in a “crew” with sequential/hierarchical process. - **AutoGen** (Microsoft, Python, MIT/CC) — conversational multi-agent, group chat, Swarm team via handoff messages. - **LangGraph multi-agent** (§1) — supervisor/network topologies as a graph.

Core abstractions to lift. Agent{instructions, tools}, Handoff(targetAgent) (transfer control + context), shared message history, supervisor/router agent, topology (star/mesh/hierarchy — already an enum in debate_orchestrator/topology).

Map onto Go + HelixCode. debate_orchestrator (consensus/dissent, AgentPool, topology enum) + llm_orchestrator (CLI-agent pool, capability matching, circuit breaker) already cover most of the *substrate*. The genuinely missing primitive is **handoff** (agent A transfers control+context to agent B, OpenAI-Swarm style) and a **supervisor/router** loop. Map handoff onto agentic. Command-style routing where a node returns the next *agent* rather than next *node*.

Distinct vs what we have / recommendation. Distinct = Swarm-style handoff + supervisor routing; we have debate (consensus) + agent pools but not handoff-as-control-transfer. → **EXTEND** debate_orchestrator (or llm_orchestrator) with a handoff + supervisor primitive rather than create a new swarm submodule (avoid duplicating the existing AgentPool/topology — §11.4.74). Mirror OpenAI Agents SDK patterns (the maintained successor), not the deprecated Swarm repo.

8. Cross-cutting integration interfaces to define (for SP6)

Define these Go interfaces so all families compose on one substrate (concurrency) and one state model (agentic.WorkflowState):

1. Executor — Run(ctx, *WorkflowState) (*Result, error) — the universal entry every model implements (agentic graph, BT, pipeline, DAG, saga, swarm).
2. Node/Step **result with routing** — NodeResult{StatePatch, GoTo []NodeID, Spawn []Node} — unifies LangGraph Command/Send, dynamic DAG mapping, and swarm handoff.
3. Scheduler — Dispatch(ready []Node) over concurrency.pool+queue with a parallelism cap + failure policy — shared by dag_orchestrator and dynamic flows.
4. Compensable — Do(ctx)/Undo(ctx) — the saga step contract.
5. Stream[T] + Operator[T, U] — the dataflow/pipeline contract.
6. Tick — Tick(ctx) (Status, error) — the behavior-tree contract; BT leaves wrap Executor.
7. Agent + Handoff — Handle(ctx, *State) (*State, *Agent, error) — swarm control-transfer.

8. Durability (**optional**) — adapter to `temporalio/sdk-go` so any flow can opt into crash-durable replay without the rest of the system depending on Temporal.
-

9. Twelve-line summary

1. Five-to-seven model families researched: dynamic-flows, dataflow/reactive, saga/compensation, behavior-trees, pipeline/map-reduce, DAG-scheduling, agent-mesh/swarm.
 2. HelixCode ALREADY has: static agentic graph, planning (HiPlan/MCTS/ToT), concurrency primitives, task-dependency data + linear executor, multi-agent pools (debate + `llm_orchestrator`).
 3. Dynamic flows → **EXTEND** `agentic` (runtime `AddNode/AddEdge + Command/Send`) + **CREATE-NEW** `flow_engine` façade; wrap Temporal Go SDK (MIT) for durability — don't reimplement replay.
 4. Dataflow/reactive + pipeline/map-reduce → **CREATE-NEW** `pipeline_runtime` (Rx-style streams + FBP components) on concurrency; vendor `RxGo/GoFlow` patterns (both Go, MIT), not full Beam.
 5. Saga/compensation → **CREATE-NEW lightweight saga primitive** in `flow_engine`; REUSE `temporalio/sdk-go` for crash-durable sagas.
 6. Behavior trees → **CREATE-NEW** `behavior_tree` submodule (vendor `askft/go-behave` MIT patterns); leaves delegate to `agentic/llm_orchestrator` handlers.
 7. DAG scheduling → **CREATE-NEW** `dag_orchestrator` scheduler over `concurrency.pool+queue + task dependency data` (no dominant Go DAG-scheduler lib exists — HONEST-GAP).
 8. Agent-mesh/swarm → **EXTEND** `debate_orchestrator/llm_orchestrator` with `handoff+supervisor` (mirror OpenAI **Agents SDK**, not deprecated Swarm); reuse existing `AgentPool/topology`.
 9. New submodules to create: `flow_engine`, `dag_orchestrator`, `pipeline_runtime`, `behavior_tree`; extend: `agentic`, `debate_orchestrator/llm_orchestrator`, `concurrency`.
 10. Top integration interfaces to define: `Executor`, `NodeResult{StatePatch, GoTo, Spawn}`, `Scheduler`, `Compensable{Do, Undo}`, `Stream/Operator`, `Tick`, `Agent/Handoff`, optional `Durability` (Temporal adapter).
 11. Reuse-not-reimplement wins (§11.4.74): Temporal Go SDK for durable execution/sagas; `RxGo/GoFlow` patterns for streams; OpenAI Agents-SDK `handoff` pattern; existing `concurrency` as the single scheduling substrate.
 12. HONEST-GAPs to resolve in SP6: confirm `joeycumines/go-behaviortree` SPDX license; decide Beam-parity scope (recommend Rx+FBP subset first); confirm `helix_code/internal/streaming` is token-streaming (not dataflow) before reusing its name.
-

Sources verified 2026-06-10

- LangGraph Graph API / StateGraph / Command / Send / conditional edges — <https://docs.langchain.com/oss/python/langgraph/graph-api> , <https://reference.langchain.com/python/langgraph/graph/state/StateGraph> , <https://langchain-ai.github.io/langgraphjs/reference/classes/langgraph.Send.html> (accessed 2026-06-10)
- Temporal Go SDK (license MIT, v1.44.1 / 2026-05-28, durable execution, saga) — <https://github.com/temporalio/sdk-go> , <https://github.com/temporalio/samples-go/blob/main/saga/workflow.go> , <https://temporal.io/blog/compensating-actions-part-of-a-complete-breakfast-with-sagas> (accessed 2026-06-10)

- RxGo (Go, MIT, pipeline-of-goroutines model) — <https://github.com/ReactiveX/RxGo> , <https://pkg.go.dev/github.com/reactivex/rxgo> (accessed 2026-06-10)
- GoFlow / FBP (Go, MIT, Component+ports) — <https://github.com/trustmaster/goflow> , <https://github.com/trustmaster/goflow/wiki/Components> (accessed 2026-06-10)
- Apache Beam Go SDK (Apache-2.0, PCollection/ParDo/bounded-unbounded/runners) — <https://beam.apache.org/documentation/programming-guide/> , <https://pkg.go.dev/github.com/apache/beam/sdks/v2/go/pkg/beam> , <https://github.com/apache/beam/blob/master/sdks/go/pkg/beam/pardo.go> (accessed 2026-06-10)
- Behavior trees Go — <https://github.com/joeycumines/go-behaviortree> , <https://github.com/askft/go-behave> (MIT) , <https://www.behaviortree.dev/groot/> (accessed 2026-06-10)
- OpenAI Swarm (Python, MIT, DEPRECATED → Agents SDK) — <https://github.com/openai/swarm> , <https://github.com/openai/openai-agents-python> (accessed 2026-06-10)
- AutoGen Swarm / CrewAI multi-agent — <https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/swarm.html> (accessed 2026-06-10)
- DAG orchestrators comparison (Airflow 3.2 / Prefect / Dagster / Flyte, dynamic task mapping) — <https://www.zenml.io/blog/orchestration-showdown-dagster-vs-prefect-vs-airflow> , <https://beam.apache.org/get-started/quickstart/go/> (accessed 2026-06-10)
- Go pipeline pattern (stage=goroutine-group over channels) — <https://go.dev/blog/pipelines> (canonical reference; accessed via RxGo/Go-pipeline corroboration 2026-06-10)

Negative / honest findings (§11.4.99(B)): (a) no single dominant Go-native general-purpose DAG-scheduler library surfaced — Go projects predominantly embed Temporal or hand-roll topo-sort+pool (justifies a new `dag_orchestrator`). (b) `joeycumines/go-behaviortree` exact SPDX license not confirmable from search snippets — verify the repo LICENSE file before vendoring. (c) OpenAI Swarm is unmaintained since 2025-03; use the **Agents SDK** patterns as the authoritative handoff reference. (d) Full Apache Beam parity is large; Rx+FBP subset recommended for first increment.